

INT102

Algorithmic Foundations and Problem Solving

参考资料

西浦学长

X



联合出品

2022.05

INT102 期末资料

INT 102 期末资料

Lec 1: Module Introduction and Pseudo Code

What is an algorithm

Some Well-known Computational Problems

Difference between Algorithm between Program

Pseudo Code 伪代码

statement

conditional

iterative (loop)

Lec 2

Time Complexity Analysis

Why efficiency matters

Big O notation

Functional Hierarchy

formal definition of Big O notation

Lec 3

Searching

Linear Search 线性查找

Binary Search 二分查找

String Matching 字符串匹配

Sorting 排序

Selection Sort 选择排序

Bubble Sort 冒泡排序

Insertion Sort 插入排序

Traveling Salesman Problem 推销员问题

Knapsack Problem 背包问题

Lec 4: Divide and Conquer 分治

Binary Search

Merge Sort 归并排序

Lec 5 Graph Theory

Graphs

Undirected graph

Representation of undirected graphs 无向图的表示方法

Directed graph

Representation of directed graphs 有向图的表示

Tree

Paths, circuits (in undirected graphs) 无向图之中的路径和闭环

Terminologies 关于树的术语:

Binary tree

Traverse a binary tree 遍历二叉树

Euler circuit 欧拉回路

How to determine whether there is an Euler circuit in a graph 如何判断欧拉回路的存在

Euler path

Hamiltonian circuit / path

Lec 6: Breadth First Search Depth First Search 深度/广度优先搜索

Breadth-first search

Depth First Search DFS

Lec 7: Greedy Methods 贪心

带权图

Greedy methods

Design of Greedy Algorithms

Minimum Spanning tree (MST) 最小生成树

- Prim's algorithm
- Kruskal's algorithm
- Lec 8: Greedy Methods (2)
 - Dijkstra's algorithm
- Lec 9: Dynamic programming in divide and conquer algorithms
 - Fibonacci numbers
 - Assembly line scheduling
- Lec 10 Space and Time Tradeoffs
 - Distribution counting sort 分布计数排序
 - Horspool's Algorithm
- Lec 11 Bellman-ford Algorithm and Floyd's Algorithm
 - Bellman-Ford algorithm
 - Floyd's Algorithm: All Pairs Shortest Paths
 - Transitive closure of a directed graph 有向图的传递闭包
- Lec12: Sequence Alignment by Dynamic Programming
 - Longest Common Subsequence
 - Pairwise Sequence Alignment Problem
- Lec 13: Introduction to Computational Complexity Theory
 - Hard Computational Problems
 - Decision/Optimisation problems
 - undecidable problems
 - Solving/Verifying a problem
- Lec 14: Introduction to Computational Complexity Theory
 - P/NP problem
 - The Class P
 - The Class NP
 - P = NP?
 - Polynomial-time reduction
 - Two Ways to Use Reductions
 - NP-hardness
 - NP-Completeness
 - NP-Complete Problem
 - Proof of NP-Completeness
- Lec 15: Coping with the Limitations of Algorithm Power
 - Exact Solution Strategies
 - Backtracking 回溯
 - n 皇后问题
 - Branch-and-Bound 分支和边界
 - The traveling salesperson optimization problem (TSP)
- Lec16: Dynamic Programming
 - 1-dimensional DP Example
 - 2-dimensional DP
 - Interval DP Example
 - Tree DP Example
 - Three step of dp
- Lec 17: Dynamic Programming (2)
 - Knapsack 0-1 Problem
- Lec18: Coping with the Limitations of Algorithm Power(2)
 - Heuristics
 - Approximation Algorithms
 - Accuracy Ratio
 - Performance Ratio
 - VERTEX-COVER 顶点覆盖
 - TSP 的一种近似解算法

Lec 1: Module Introduction and Pseudo Code

What is an algorithm

- A sequence of precise and concise instructions that guide you (or a computer) to solve a class of specific problem

算法是指导你(人)或者计算机完成一类问题的精确而简洁的指令。



Some Well-known Computational Problems

- Sorting
- Searching
- Graph and Combinatorial Problems
 - Shortest paths in a graph
 - Minimum spanning tree
 - Traveling salesman problem
 - Knapsack problem
- Problem in Number Theory
 - Primality testing

Difference between Algorithm between Program

- Algorithms are free from grammatical rules
 - Content is more important than form
 - Acceptable as long as it tells people how to perform a task
- Programs must follow some syntax rules
 - Form is important
 - Even if the idea is correct, it is still not acceptable if there is syntax error

Pseudo Code 伪代码

尽管在表示算法时，形式并不重要，但是我们仍需要一套规范来确保交流的效率，这就是伪代码 (Pseudo Code), 伪代码比可执行的代码更接近自然语言。

statement

```
variable = expression
```

声明代码块:

```
begin
  statement1
  statement2
end
```

conditional

此处的 condition 为语句执行的条件:

```
if condition then  
    statement
```

iterative (loop)

for 循环的步进默认为 1:

```
for var = start_value to end_value do  
    statement
```

此处的 condition 为循环继续的条件:

```
while condition do  
    statement
```

此处的 condition 为循环结束的条件:

```
repeat  
    statement  
until condition
```

Lec 2

Time Complexity Analysis

如何衡量一个算法有多快:

- identify some important operations/steps and count how many times these operations/steps needed to be executed.
- Number of operations usually expressed in terms of input size.
- how does the number of steps of operations grow when the input size grows (If we doubled the input size, how much longer would the algorithm take? If we trebled the input size, how much longer would it take?)

Why efficiency matters

- speed of computation by hardware has been improved
- efficiency still matters
- ambition for computer applications grows with computer power
- demand a great increase in speed of computation

Big O notation

为了比较算法之间的性能, 我们引入大O记法 (Big O notation)。

观察下五个方程的值随着输入增大而增大的幅度。

$$f_1(n) = 50n + 20$$

$$f_2(n) = 10 n \log_2 n + 100$$

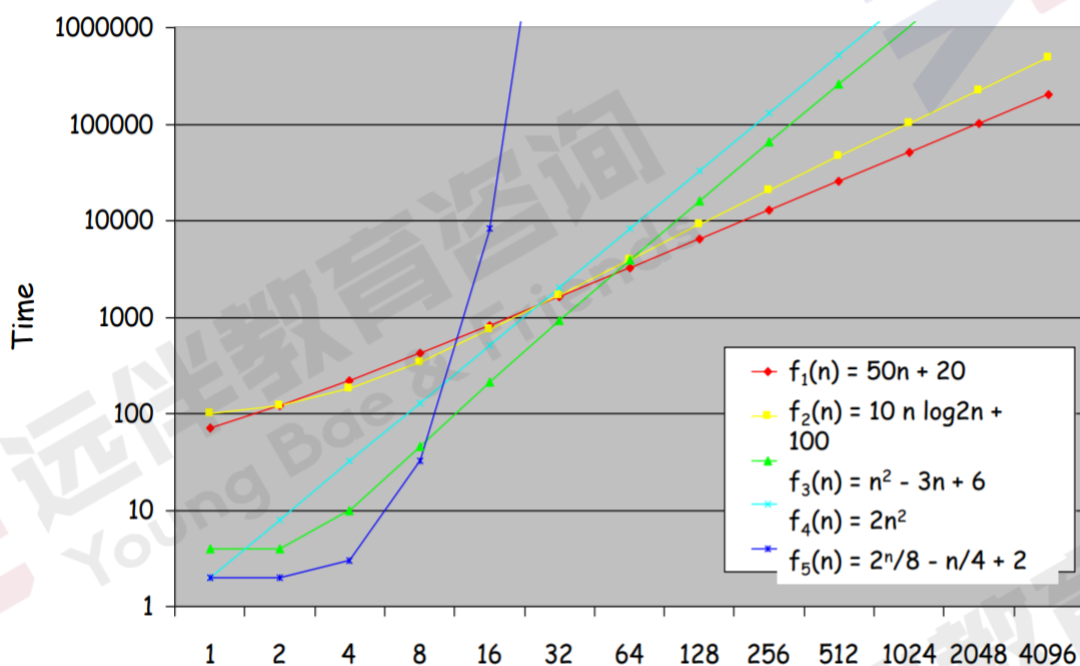
$$f_3(n) = n^2 - 3n + 6$$

$$f_4(n) = 2n^2$$

$$f_5(n) = 2^n/8 - n/4 + 2$$

n	1	2	4	8	16	32	64	128	256	512	1024	2048
$f_1(n) = 50n + 20$	70	120	220	420	820	1620	3220	6420	12820	25620	51220	102420
$f_2(n) = 10 n \log_2 n + 100$	100	120	180	340	740	1700	3940	9060	20580	46180	102500	225380
$f_3(n) = n^2 - 3n + 6$	4	4	10	46	214	934	3910	16006	64774	3E+05	1E+06	4E+06
$f_4(n) = 2n^2$	2	8	32	128	512	2048	8192	32768	131072	5E+05	2E+06	8E+06
$f_5(n) = 2^n/8 - n/4 + 2$	2	2	3	32	8190	5E+08	2E+18					

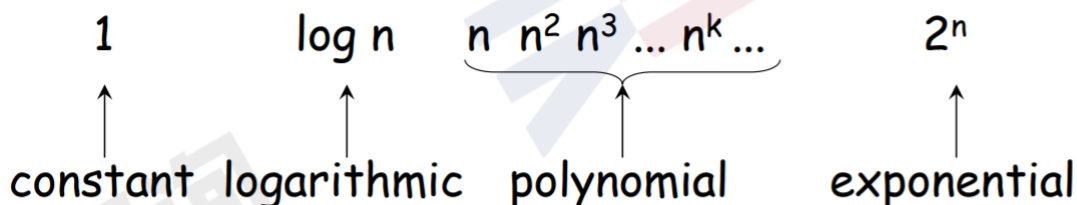
图像看起来更加直观：



可以看到，随着输入的增长有有些函数增长的快，有些函数增长的慢，这就是 **Functional Hierarchy**

Functional Hierarchy

We can define a hierarchy of functions each having a greater order of magnitude than its predecessor:



We can further refine the hierarchy by inserting $n \log n$ between n and n^2 , $n^2 \log n$ between n^2 and n^3 , and so on.

as we move from left to right, successive functions have greater order of magnitude than the previous ones. As n increases, the values of the later functions increase more rapidly than the values of the earlier ones.

简单来说，一个函数的 **growth rate** 在无穷大处由其中数量级(magnitude)最大的项决定：

For example, $f(n) = 2n^3 + 5n^2 + 4n + 7$ The term with the highest power is $2n^3$. The growth rate of $f(n)$ is dominated by n^3 .

大O记法的工作原理就是只关心函数中决定函数增长率的项。

formal definition of Big O notation

$f(n) = O(g(n))$: There exists a constant c and n_0
such that $f(n) \leq c g(n)$ for all $n > n_0$

数学语言记法:

$\exists$ constant c and n_0 such that $\forall n > n_0, f(n) \leq cg(n)$

value of $f(n)$ for $n < n_0$ does not matter

Usually we are only interested in the asymptotic time complexity, i.e., when n is large

也就是 n 趋向于无限大时

$$O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n)$$

上面6种大O记法基本就是我们现阶段会遇到所有可能的大O记法了

Lec 3

Searching

Input: a sequence of n numbers $a_0, a_1, \dots, a_{n-1}$; and a number X .

Output: determine whether X is in the sequence or not.

Linear Search 线性查找

遍历输入，在每次遍历中比较目标值和当前遍历到的值，如果相等则输出 "Found!" 并且退出程序，如果循环结束还没出现相等的情况，输出 "Not Found!"，退出程序。

```
i = 0
while i < n do
begin
    if x == a[i] then
        report "Found!" and stop
    else
        i = i+1
end
report "Not Found!"
```

在搜索算法中，一个最重要也最基本的操作就是 **comparison**，比较算法的性能可以用大 O 记法表示的算法所需的比较次数来描述。

一个搜索算法找到目标值所需的比较次数除了和自身性能有关，也和输入有关，因此存在最坏情况和最好情况

Best case: X is the 1st no., 1 comparison, $O(1)$

Worst case: X is the last no. OR X is not found, n comparisons, $O(n)$

Binary Search 二分查找

more efficient way of searching when the sequence of numbers is pre-sorted

Input: a sequence of n sorted numbers $a_0, a_1, \dots, a_{n-1}$ in ascending order and a number X

Idea of algorithm:

- compare X with number in the middle
- then focus on only the first half or the second half (depend on whether X is smaller or greater than the middle number)

```
first=0, last=n-1
while (first <= last) do
begin
    mid = floor((first+last)/2)
    if (X == a[mid])
        report "Found!" & stop
    else
        if (X < a[mid])
            last = mid-1
        else
            first = mid+1
end
report "Not Found!"
```

Best case: X is the number in the middle $\Rightarrow$ 1 comparison, $O(1)$ -time

Worst case: at most $\lceil \log_2 n \rceil + 1$ comparisons, $O(\log n)$ time.

String Matching 字符串匹配

Given a string of n characters called the text and a string of m characters ($m \leq n$) called the pattern.

We want to determine if the text contains a substring matching the pattern.

这里的字符串匹配算法是线性搜索算法的扩展，在查找长度为 m 的字串时，在每次遍历时嵌套一个循环用来检查以当前遍历的字符开头以及其后面 $m-1$ 位的字符构成的字符串是否是目标字符串。

The algorithm scans over the text position by position. For each position i , it checks whether the pattern $P[0..m-1]$ appears in $T[i..i+m-1]$

If the pattern exists, then report found Else continue with the next position $i+$. If repeating until the end without success, report not found

```
for i = 0 to n-m do
begin
    j = 0
    while (j < m && P[j]==T[i+j]) do
        j = j + 1
    if (j == m) then
        report "found!" & stop
end
report "Not found!"
```

Best case: pattern appears in the beginning of the text, $O(m)$ -time

Worst case: pattern appears at the end of the text OR pattern does not exist, $O(nm)$ -time

Sorting 排序

Input: a sequence of n numbers $a_0, a_1, \dots, a_{n-1}$

Output: arrange the n numbers into ascending order, i.e., from smallest to largest

Selection Sort 选择排序

1. find minimum key from the input sequence
2. delete it from input sequence
3. append it to resulting sequence
4. repeat until nothing left in input sequence

找到原数组的最小值，将其拼到新数组的尾部并将其从原数组中删除，直到原数组用尽。

```
for i = 0 to n-2 do
begin
    min = i
    for j = i+1 to n-1 do
        if a[j] < a[min] then
            min = j
    swap a[i] and a[min]
end
```

Total number of comparisons $= (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

选择排序的时间复杂度时确定的，不存在最好情况和最差情况的说法。

Time Complexity: $O(n^2)$

Bubble Sort 冒泡排序

1. starting from the last element, swap adjacent items if they are not in ascending order.
2. when first item is reached, the first item is the smallest.
3. repeat the above steps for the remaining items to find the second smallest item, and so on.

```
for i = 0 to n-2 do
    for j = n-1 downto i+1 do
        if (a[j] < a[j-1])
            swap a[j] & a[j-1]
```

冒泡排序重复地走访过要排序的数列，一次比较两个元素，如果他们的顺序错误就把他们交换过来。走访数列的工作是重复地进行直到没有再需要交换，也就是说该数列已经排序完成。这个算法的名字由来是因为越小的元素会经由交换慢慢“浮”到数列的顶端

Total number of comparisons $= (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

Time complexity: $O(n^2)$

Insertion Sort 插入排序

1. look at elements one by one
2. build up sorted list by inserting the element at the correct location

```
for i = 1 to n-1 do
begin
    key = a[i]
    pos = 0
    while (a[pos] < key) && (pos < i) do
        pos = pos + 1
    shift a[pos], ..., a[i-1] to the right
    a[pos] = key
end
```

total number of comparisons $= 1 + 2 + \dots + n - 1 = \frac{(n-1)}{n^2}$

Time complexity: $O(n^2)$

Traveling Salesman Problem 推销员问题

这个问题仅仅是指数时间复杂度问题的例子，此处不要求理解和掌握，只要记住推销员问题可以是指数时间复杂度即可。

Input: There are n cities.

Output: Find the shortest tour from a particular city that visit each city exactly once before returning to the city where it started (Hamiltonian circuit).

A Hamiltonian circuit can be represented by a sequence of $n+1$ cities $v_1, v_2, \dots, v_n, v_1$, where the first and the last are the same, and all the others are distinct.

Exhaustive search approach: Find all tours in this form, compute the tour length and find the shortest among them

按照上面这种思路，一共要考虑 $(n-1)! = (n-1)(n-2)\dots 1$ 种不同的路线（指数时间复杂度）。

Knapsack Problem 背包问题

这个问题仅仅是指数时间复杂度问题的例子，此处不要求理解和掌握，只要记住背包问题可以是指数时间复杂度即可。

Input: Given n items with weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$, and a knapsack with capacity W .

Output: Find the most valuable subset of items that can fit into the knapsack.

Application: A transport plane is to deliver the most valuable set of items to a remote location without exceeding its capacity

Exhaustive search approach: Try every subset of the set of n given items, compute total weight of each subset and compute total value of those subsets that do NOT exceed knapsack's capacity

按照上面这种思路，一共要考虑 2^n 种不同的路线（指数时间复杂度）。

Lec 4: Divide and Conquer 分治

分治是最知名的算法设计技巧之一，其核心思想为：

- A problem instance is divided into several smaller instances of the same problem, ideally of about same size
- The smaller instances are solved, typically recursively
- The solutions for the smaller instances are combined to get a solution to the original problem

简单来说，想要对一个问题应用分治思想，需要思考一下问题：

1. 基准情况是什么 (base case)?
2. 如何把父问题通过**相同的递进逻辑**分解成子问题？（也就是在将问题分解到基准情况之前可以一直分下去）
3. 一个父问题所有子问题的解决是否等价于父问题的解决？

从以上三点可以看出，分治的思想和递归的逻辑是高度契合的，因此实现分治最方便的办法就是递归。

Binary Search

Input: a sequence of n sorted numbers $a_0, a_1, \dots, a_{n-1}$; and a number X

Idea of algorithm:

- compare X with number in the middle
- then focus on only the first half or the second half (depend on whether X is smaller or greater than the middle number)
- reduce the amount of numbers to be searched by half

```
RecurBinarySearch(A, first, last, X)
begin
    if (first > last) then
        return false
    mid = floor((first + last)/2)
    if (X == A[mid]) then
        return true
    if (X < A[mid]) then
        return RecurBinarySearch(A, first, mid-1, X)
    else
        return RecurBinarySearch(A, mid+1, last, X)
end
```

Time complexity:

采用了分治思想的算法，其时间复杂度可以用下面这种方法证明：

Let $T(n)$ denote the time complexity of binary search algorithm on n numbers.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + 1 & \text{otherwise} \end{cases}$$

以上这个式子可以理解为时间复杂度函数的递推公式 (recurrence)

所有蕴含分治思想的算法，其复杂度函数都可以写出形如上面这个式子的递推公式。

证明的第一步是假设，一般通过观察算法实现或者伪代码就已经能看出算法的大O记法复杂度。一般来说蕴含二分思想的算法复杂度为 $O(\log n)$ 一些更复杂的算法可能是 $O(n \log n)$ 。

有了 $\log n$ 这个目标，就可以开始假设了，这里我们假设：

$$T(n) \leq 2 \log n \quad (1)$$

不等式右边这一项乘上常数系数，或者加减常数都不会影响最终的结果（想一想大O记法的性质），因此可以通过增加系数，加减常数来简化证明，这个系数或者加减的常数一般是在证明的过程中发现的，这里直接假设 $2 \log n$ 是完成了证明之后才知道的最佳做法。

如果你的目标函数数量级很高，也可以加减数量级比目标函数低的函数式。

证明的顺序不是思考的顺序。

$n = 1$ 时， $T(1) = 1$ 这里将其排除在复杂度的考虑之外。（大O记法的只关注n较大时的值）

$n = 2$ 时

$$L.H.S = T(2) = T(1) + 1 = 2$$

$$R.H.S = 2 \log 2 = 2$$

$$L.H.S \leq R.H.S$$

$n > 2$ 时

$$T(n) = T\left(\frac{n}{2}\right) + 1, \text{ 由 (1) 得 } T\left(\frac{n}{2}\right) + 1 \leq 2 \log\left(\frac{n}{2}\right) + 1$$

$$2 \log\left(\frac{n}{2}\right) + 1 = 2(\log n - 1) + 1 = 2 \log n - 1$$

很显然, $2 \log n - 1 < 2 \log n$

因此 $n \geq 2$ 时, $T(n) \leq 2 \log n$

因为此算法的时间复杂度 $T(n) \leq 2 \log n$ ，其时间复杂度可以用 $O(\log n)$ 表示。

以下是关于蕴含分治思想的算法复杂度的经验性总结，得出递推公式后，就可以根据下面的式子大胆地进行假设了：

$$T(n) = T(n/2) + 1 \text{ } T(n) \text{ is } O(\log n)$$

$$T(n) = 2T(n/2) + 1 \text{ } T(n) \text{ is } O(n)$$

$$T(n) = 2T(n/2) + n \text{ } T(n) \text{ is } O(n \log n)$$

Merge Sort 归并排序

Idea of algorithm:

Divide: dividing a sequence of n numbers into two smaller sequences is straightforward.

Conquer: merging two sorted sequences of total length n can also be done easily, at most $n - 1$ comparisons.

Analysis:

想要理解归并排序，首先要理解：如何将两个有序数组合并为一个有序数组：

- To merge two sorted sequences, we keep two pointers, one to each sequence.
- Compare the two numbers pointed, copy the smaller one to the result and advance the corresponding pointer.
- Then compare again the two numbers pointed to by the pointer; copy the smaller one to the result and advance that pointer.
- Repeat until both sequences are used up.

简单来说，两个数组都从左往右一位一位的看，两个数组那个最左边的一位最小，就把这一位拼到结果数组尾部，并将其从原数组中删除，直到两个数组都用尽。

理解了这一点之后再看归并排序就简单了。归并排序其实就是将数组不停地二分直到每一份都是有序的（只含有一个元素的数组天然有序），然后再将有序数组两两合并，一层层合并出最终的结果。

```
Algorithm Mergesort(A[0..n-1])
  if n > 1 then begin
    copy A[0..floor(n/2)-1] to B[0..floor(n/2)-1]
    copy A[floor(n/2) .. n-1] to C[0..floor(n/2)-1]
    Mergesort(B[0..floor(n/2)-1])
    Mergesort(C[0..floor(n/2)-1])
    Merge(B, C, A)
  end
```

归并排序的时间复杂度递推公式为:

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + n & \text{otherwise} \end{cases}$$

根据上面的经验公式，归并排序的复杂度为 $O(n \log n)$ ，此处不再证明。

Lec 5 Graph Theory

Graphs

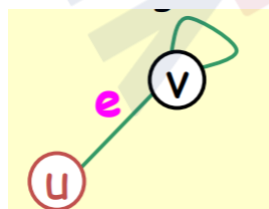
An undirected graph $G=(V,E)$ consists of a set of vertices V and a set of edges E . Each edge is an unordered pair of vertices

A directed graph $G=(V,E)$ consists of ... Each edge is an ordered pair of vertices.

Undirected graph

- simple graph: at most one edge between two vertices, no self loop (i.e., an edge from a vertex to itself).
- multigraph: allows more than one edge between two vertices.
- pseudograph: allows a self loop

下面根据这张样图介绍无向图中的一些概念:



- u and v are said to be adjacent and called neighbors of each other. u和v相邻，并是彼此的邻居。
- u and v are called endpoints of e. u和v被称作 e 的端点。
- e is said to be incident with u and v. e 与u和v有关。
- e is said to connect u and v.
- The degree of a vertex v, denoted by $\deg(v)$, is the number of edges incident with it. 一个顶点的度(degree)就是和这个顶点有关的边的数量。

Representation of undirected graphs 无向图的表示方法

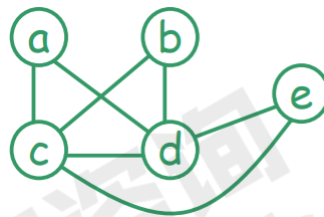
An undirected graph can be represented by:

- adjacency matrix 邻接矩阵
- adjacency list 邻接表
- incidence matrix or incidence list. 关联矩阵或关联列表

Adjacency matrix and adjacency list record the relationship between vertex adjacency, i.e., a vertex is adjacent to which other vertices

Incidence matrix and incidence list record the relationship between edge incidence, i.e., an edge is incident with which two vertices

以下面这个图为例讲解图的表示方法



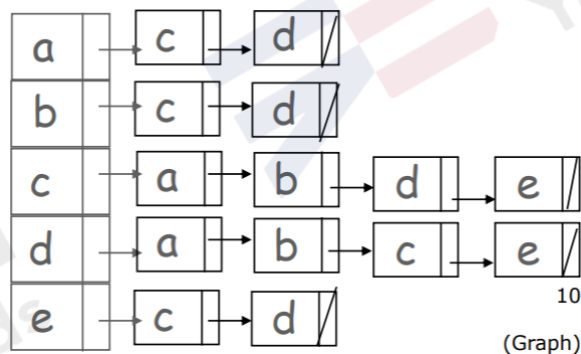
Adjacency matrix:

邻接矩阵是一个方阵，每一行和每一列都依次代表一个图中的节点，如果代表两个节点行和列的相交处的矩阵元素为 0 代表这两个节点没有被相连，为 1 则代表这两个节点被相连。

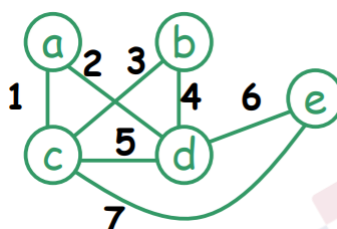
	a	b	c	d	e
a	0	0	1	1	0
b	0	0	1	1	0
c	1	1	0	1	1
d	1	1	1	0	1
e	0	0	1	1	0

Adjacency list:

邻接表包含和顶点数量相同数量的链表，每个链表上的头元素以外的元素都和头元素有边相连。



Incidence matrix:



M for a simple undirected graph with n vertices and m edges:

- M is an $m \times n$ matrix
- $M(i, j) = 1$ if edge i and vertex j are incidence

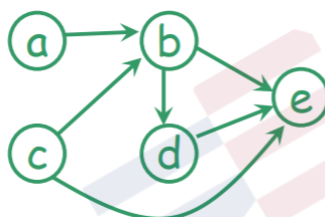
$$\begin{matrix}
 & \begin{matrix} a & b & c & d & e \end{matrix} \\
 \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix}
 \end{matrix}$$

Incidence list:

each edge has a list of vertices to which it is incident with

1		a	c
2		a	d
3		b	c
4		b	d
5		c	d
6		d	e
7		c	e

Directed graph



Given a directed graph G , a vertex a is said to be connected to a vertex b if there is a path from a to b .

下面根据这张样图介绍有向图中的一些概念:

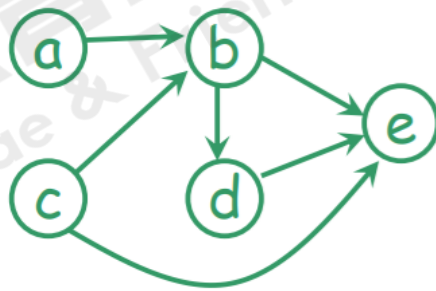
入度和出度:

The in-degree of a vertex v is the number of edges leading to the vertex v .

The out-degree of a vertex v is the number of edges leading away from the vertex v

其他的概念基本和有向图相同

Representation of directed graphs 有向图的表示



Similar to undirected graph, a directed graph can be represented by

- adjacency matrix
- adjacency list
- incidence matrix or incidence list.

adjacency matrix:

$$\begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

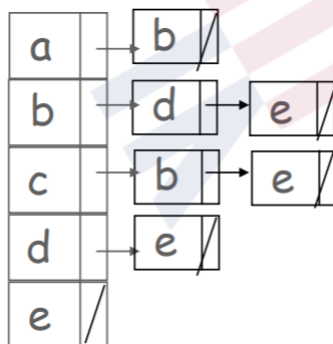
Adjacency matrix M for a directed graph with n vertices:

- M is an $n \times n$ matrix
- $M(i, j) = 1$ if (i, j) is an edge

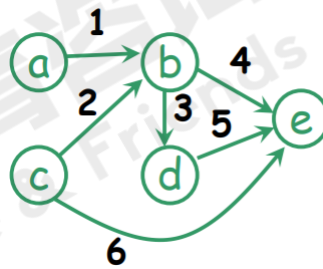
有向图的邻接矩阵中，存在从行代表的节点指向列代表的节点的边，行列交叉处才为1。

Adjacency list:

- each vertex u has a list of vertices pointed to by an edge leading away from u



Incidence matrix:



M for a directed graph with n vertices and m edges is an $m \times n$ matrix

- $M(i, j) = 1$ if edge i is leading away from vertex j
- $M(i, j) = -1$ if edge i is leading to vertex j

$$M = \begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 0 & -1 \end{pmatrix} \end{matrix}$$

Incidence list:

Each edge has a list of two vertices (leading away is 1st and leading to is 2nd)

1	a	b
2	c	b
3	b	d
4	b	e
5	d	e
6	c	e

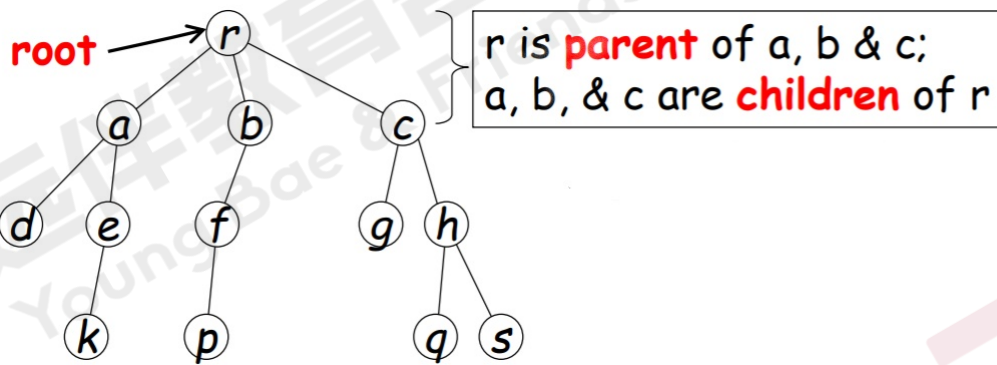
Tree

Paths, circuits (in undirected graphs) 无向图之中的路径和闭环

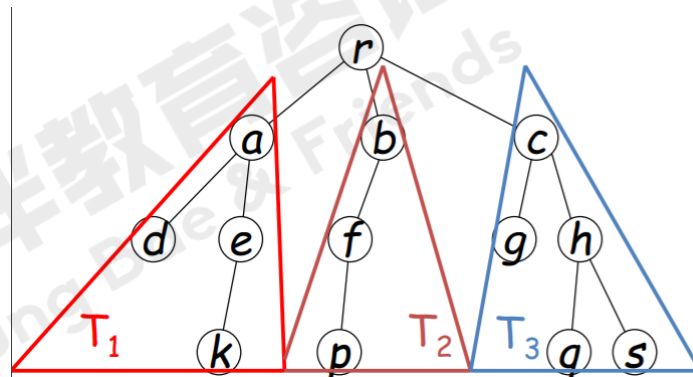


- In an undirected graph, a path from a vertex u to a vertex v is a sequence of edges $e_1 = \{u, x_1\}$, $e_2 = \{x_1, x_2\}$, ..., $e_n = \{x_{n-1}, v\}$, where $n \geq 1$.
- The length of this path is n .
- Note that a path from u to v implies a path from v to u .
- If $u = v$, this path is called a circuit (cycle).
- If every pair of vertices have a path, then Graph is called connected

Terminologies 关于树的术语:



- Topmost vertex is called the root.
- A vertex u may have some children directly below it, u is called the parent of its children. □
- Degree of a vertex is the no. of children it has. (N.B. it is different from the degree in an unrooted tree.)
- Degree of a tree is the max. degree of all vertices.
- A vertex with no child (degree-0) is called a leaf. All others are called internal vertices.

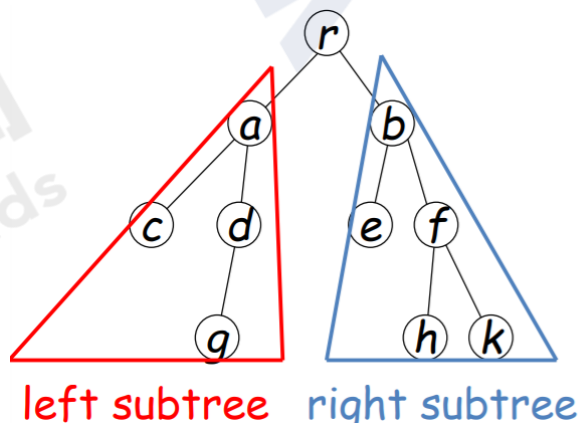


We can define a tree recursively:

- A single vertex is a tree.
- If $T_1, T_2, \dots, T_k$ are disjoint trees with roots $r_1, r_2, \dots, r_k$, the graph obtained by attaching a new vertex r to each of $r_1, r_2, \dots, r_k$ with a single edge forms a tree T with root r .
- $T_1, T_2, \dots, T_k$ are called subtrees of T .

Binary tree

- 二叉树指的是一颗度最大为2的树。
- 这个棵树的两棵子树被称为左子树和右子树。



Traverse a binary tree 遍历二叉树

树的一个显著特点就是无法仅仅根据子树追溯根 (root), 因此想要遍历整个二叉树就需要一定的章法, 否则会因为引用丢失无法回到上层引用。

There are three common ways to traverse a binary tree:

由于树可以是递归地定义的, 这三种遍历方法都天然可以用递归实现。

当然, 用迭代也可以实现, 虽然不如递归实现直观, 但是性能高于递归实现。

- preorder traversal - vertex, left subtree, right subtree

```
PreorderTraversal (vertex)
  //do something (eg. print the value of vertex if exists)
  PreorderTraversal(left child of vertex)
  PreorderTraversal(right child of vertex if exists)
end
```

- inorder traversal - left subtree, vertex, right subtree

```
InorderTraversal (vertex)
  InorderTraversal(left child of vertex if exists)
  //do something (eg. print the vertex)
  InorderTraversal(right child of vertex if exists)
end
```

- postorder traversal - left subtree, right subtree, vertex

```
PostorderTraversal (vertex)
  PostorderTraversal(left child of vertex if exists)
  PostorderTraversal(right child of vertex)
  //do something (eg. print the vertex if exists)
end
```

Euler circuit 欧拉回路

Euler circuit: A simple circuit visits an edge at most once.

An Euler circuit in a graph G is a circuit visiting every edge of G exactly once. (A vertex can be repeated.)

著名的七拱桥问题其实就是寻找一个无向图中的欧拉回路的问题

How to determine whether there is an Euler circuit in a graph 如何判断欧拉回路的存在

首先谈论一种特殊情况, 如果一个无向图的每对顶点之间都有一条路径(path), 则称无向图 G 是连通的 (connected)

如果一个无向图是**不连通的**, 那么其必然**不存在欧拉电路**

下面在确保图连通的情况下谈论更加普遍的情况 (**Necessary and sufficient condition**):

Let G be a connected graph.

Lemma(引理): G contains an Euler circuit if and only if every vertex has even degree.

G 是一个连通图, 当且仅当 G 的每个顶点的度都为偶数是, G 才包含一个欧拉回路。

Euler path

Let G be an undirected graph.

An Euler path is a **path** visiting every edge of G **exactly once**.

An undirected graph contains an Euler path if it is connected and contains exactly two vertices of odd degree.

path 的要求比 circuit 容易满足，是因为 path 不需要首尾相接形成闭环。

一个不包含欧拉回路的图可能包含欧拉路径。



以上这个图的欧拉路径为 c b d a c e d

	Exist Euler circuit?	Exist Euler path?
all vertices have even degree	YES	YES
exactly two vertices have odd degree	NO	YES
more than 2 vertices have odd degree	NO	NO

Hamiltonian circuit / path

Let G be an undirected graph.

- A Hamiltonian circuit (path) is a circuit (path) containing every vertex of G exactly once.
- Note that a Hamiltonian circuit or path may NOT visit all edges.
- Unlike the case of Euler circuits / paths, determining whether a graph contains a Hamiltonian circuit (path) is a very difficult problem. (NP-hard)

哈密顿回路是在一个图中每个顶点只到过一次的回路。

寻找哈密顿回路问题是一个NP问题。

Lec 6: Breadth First Search Depth First Search 深度/广度优先搜索

想要开始遍历一张图，首先要选定一个入口（入口可以任意选择），基于这个入口顶点，深度和广度这两个词的具体含义：

深度：从入口出发形成一条路径 (path)，这个**路径的长度**就是深度 Depth。

广度：从入口出发形成一个放射网，从当一个节点放射出的**边的数量**就是广度 Breadth。

立足于图中的一个节点，需要从所有的相邻的节点中选择下一个前往的节点，深度优先和广度优先本质上就是不同的选择策略。

深度优先也就是指 "如果所有可达节点中有可以使深度增加的就增加深度，如果没有了就回到深度少1的相邻节点增加一次广度"，而广度优先则相反 "如果深度比自己少1的相邻节点的广度还没有达到最大值，就回到那个节点去增加他的广度，否则增加一次深度"。

深度优先和广度优先的选择策略不足以做出所有决定，大部分情况下我们还需要另外的辅助策略。例如在深度优先搜索中，如果当前节点有三个相邻的更深的节点，此时如何选择？这个辅助策略就是根据节点的其他属性做出选择的策略，由具体的问题需求决定，常见的辅助策略有按节点名称字母顺序，或者按节点数值大小。

Breadth-first search

All vertices at distance k from s are explored before any vertices at distance $k+1$

A simple algorithm for searching a graph. Given $G=(V, E)$, and a distinguished source vertex s , BFS systematically explores the edges of G such that

- all vertices at distance k from s are explored before any vertices at distance $k+1$

```
unmark all vertices
choose some starting vertex s
mark s and insert s into tail of list L
while L is nonempty do
begin
    remove a vertex v from front of L
    visit v
    for each unmarked neighbor w of v do
        mark w and insert w into tail of list L
end
```

Depth First Search DFS

Edges are explored from the most recently discovered vertex, backtracks when finished

Depth-first search is another strategy for exploring a graph; it search "deeper" in the graph whenever possible.

- Edges are explored from the most recently discovered vertex v that still has unexplored edges leaving it.
- When all edges of v have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.

```
unmark all vertices
choose some starting vertex s
mark s and insert s into tail of list L
while L is nonempty do
begin
    read the last element v of list L as v
    visit v
    if v has neighbour
    begin
        visit one of v's neighbour and insert that neighbour into tail of list L
    end
    else
    begin
        remove the last element of list L
    end
end
```


Lec 7: Greedy Methods 贪心

有了图 (Graph) 的概念之后，基于图衍生出了一个问题，就是**路径规划问题**，想要理解路径规划问题，首先要引入一个概念：

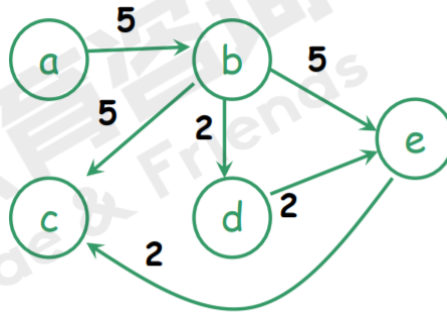
带权图

无权图的定义是由一个边集合和一个顶点集合构成的，带权图在此基础上给每条边一个权重 (weight)。

一个以城市为顶点，之间的主干道为边，距离为边的权重的图就是一个典型的带权图。

Consider a (un)directed connected graph G

- The edges are labelled by weight
- Given a particular vertex called the source
 - Find shortest paths from the source to all other vertices (shortest path means the total weight of the path is the smallest)



Greedy methods

- How to be greedy?
 - At every step, make the best move you can make.
 - Keep going until you're done
- Advantages
 - Don't need to pay much effort at each step.
 - Usually finds a solution very quickly.
 - The solution found is usually not bad.
- Possible problem
 - The solution found may NOT be the best one

Design of Greedy Algorithms

- Cast the optimization problem as one in which we make a choice and are left with one subproblem to solve.
- Demonstrate that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.

Greedy methods - examples:

- Minimum spanning tree
 - Prim's algorithm
 - Kruskal's algorithm
- Single-source shortest-paths

- Dijkstra's algorithm
- All above-mentioned algorithms find (one of) the BEST solution

Minimum Spanning tree (MST) 最小生成树

- Given an undirected connected graph G
 - The edges are labelled by weight
- Spanning tree of G
 - a tree containing all vertices in G
- Minimum spanning tree
 - a spanning tree of G with minimum weight

Prim's algorithm

Theorem: Let $G=(V,E)$ be a connected, undirected graph with a real-valued weights on edges. Let A be a subset of E that is included in some minimal spanning tree for G , let S be a subset of V and contain V_A , where $V_A = \{v : \text{there is an edge } e = (u,v) \text{ in } A\}$. If (u^*, v^*) is one of the edge with minimum weight among the edges connecting S and $V-S$, then $A \cup (u^*, v^*)$ is contained in a minimum spanning tree.

Pseudo code:

```
// Given a weighted connected graph  $G=(V,E)$ 
pick a vertex  $v_0$  in  $V$ 
 $V_T = \{v_0\}$ 
 $E_T = \emptyset$ 
For  $i=1$  to  $|V|-1$  do
    pick an edge  $e=(v^*, u^*)$  with minimum weight
    among all the edges  $(v, u)$  such that  $v$  is in  $V_T$  and  $u$ 
    is in  $V - V_T$ 
     $V_T = V_T \cup \{v^*\}$ 
     $E_T = E_T \cup \{e^*\}$ 
Return  $E_T$ 
```

简单来说 Prim's algorithm 就是每次从边集中取出以已经用过的顶点为起点，还未走过的顶点做终点的边中最短的一条放入结果边集合中，直到所有的顶点都被用过。

1. 输入：一个加权连通图，其中顶点集合为 V ，边集合为 E ；
2. 初始化： $V_{\text{new}} = \{x\}$ ，其中 x 为集合 V 中的任一节点（起始点）， $E_{\text{new}} = \{\}$ ；
3. 重复下列操作，直到 $V_{\text{new}} = V$ ：
 1. 在集合 E 中选取权值最小的边 (u, v) ，其中 u 为集合 V_{new} 中的元素，而 v 则是 V 中没有加入 V_{new} 的顶点（如果存在有多条满足前述条件即具有相同权值的边，则可任意选取其中之一）；
 2. 将 v 加入集合 V_{new} 中，将 (u, v) 加入集合 E_{new} 中；
4. 输出：使用集合 V_{new} 和 E_{new} 来描述所得到的最小生成树。

Correctness Analysis:

当开始学习越来越复杂的算法时，这些算法的正确性就没有那么一目了然了，因此在学习较为复杂的算法时就需要分析其正确性。

对于 Prim's algorithm，其正确性的关键在于其生成的树一定是最小生成树吗？

通过观察上面的定理以及伪代码我们可以逆推，在每次循环开始之前， E_T 都是最终的最小生成树的子集，因此这个算法的合理性应当可以通过数学归纳法(mathematical induction)证明，这里就不展开了。

Complexity of Prim's Algorithm

在上面的伪代码中提到的

pick an edge $e = (v^, u^*)$ with minimum weight among all the edges (v, u) such that v is in V_T and u is in $V - V_T$*

这一步，其具体的代码实现是基于最小堆 min-heap，或者说优先队列 priority queue：

```

MST-PRIM( $G, w, \text{root}$ )  //  $G=(V, E)$ 
1.  for each  $u$  in  $V$ 
2.      do  $\text{key}[u] \leftarrow \infty$ 
3.      do  $\pi[u] \leftarrow \text{NIL}$ 
4.   $\text{key}[\text{root}] \leftarrow 0$ 
5.   $Q \leftarrow V$  // implement by min-heap,  $O(|V|)$ 
6.  while  $Q$  is not empty
7.      do  $u \leftarrow \text{Extract-min}(Q)$  //  $O(\lg(|V|))$  by heap
8.      for each  $v$  in  $\text{Adj}[u]$ 
9.          do if  $v$  in  $Q$  and  $w(u, v) < \text{key}[v]$ 
10.             then  $\pi[v] \leftarrow u$ 
11.              $\text{key}[v] = w(u, v)$ 
  
```

没学过 CPT102 的同学可能不知道最小堆，此处的数据结构细节本课程不要求掌握。只需要直到上面代码的第7行这一步得到的 u 就是以已经用过的顶点为起点，还未走过的顶点做终点的边中最短的一条。

基于这个较为具体的实现，Prim's algorithm 的时间复杂度为 $O(|E|\log |V|)$

$|E|$ 是图中边的数量， $|V|$ 则为节点数量

Kruskal's algorithm

Kruskal's algorithm is greedy in the sense that it always attempt to select the smallest weight edge to be included in the MST

Pseudo code

```
// Given an undirected connected graph  $G=(V,E)$ 
pick an edge  $e$  in  $E$  with minimum weight
 $T = \{ e \}$  and  $E' = E - \{ e \}$ 
while  $E' \neq \emptyset$  do
  begin
    pick an edge  $e$  in  $E'$  with minimum weight
    if adding  $e$  to  $T$  does not form cycle then
       $T = T \cup \{ e \}$ 
       $E' = E' - \{ e \}$ 
  end
```

Kruskal's algorithm 比 Prim's algorithm 理解起来简单一些。Kruskal's algorithm 就是每次选取权值最小的边加入结果集中，如果结果集中没有成环就继续这个操作。如果加入某一条边会使结果集成环，那么就将加入这条边之前的结果集输出。

Kruskal's algorithm 的时间复杂度也为 $O(|E|\log |V|)$

Lec 8: Greedy Methods (2)

Single-source shortest-paths

- Consider a (un)directed connected graph G
 - The edges are labelled by weight
- Given a particular vertex called the source
 - Find shortest paths from the source to all other vertices (shortest path means the total weight of the path is the smallest)

Optimal substructure of shortestpaths

- Optimal substructure of a shortestpath
 - A shortest path between two vertices contains other shortest paths within it.
- Algorithms
 - there are many algorithms to solve this problem, one of them is Dijkstra's algorithm, which assumes the weights of edges are non-negative

Dijkstra's algorithm

Input: A directed connected weighted graph G and a source vertex s

Output: For every vertex v in G , find the shortest path from s to v

Dijkstra's algorithm runs in iterations:

- in the i -th iteration, the vertex which is the i -th closest to s is found
- for every remaining vertices, the current shortest path to s found so far (this shortest path will be updated as the algorithm runs)

To describe the algorithm using pseudo code, we give some notations.

- Each vertex v is labelled with two labels:
 - a numeric label $d(v)$ indicates the length of the shortest path from the source to v found so far
 - another label $p(v)$ indicates next-to-last vertex on such path, i.e., the vertex immediately before v on that shortest path

Pseudo code:

```
// Given a graph  $G=(V,E)$  and a source vertex  $s$ 
for every vertex  $v$  in the graph do
    set  $d(v) = \infty$  and  $p(v) = \text{null}$ 
set  $d(s) = 0$  and  $V_T = \emptyset$ 
while  $V - V_T \neq \emptyset$  do    // there is still some vertex left
begin
    choose the vertex  $u$  in  $V - V_T$  with minimum  $d(u)$ 
    set  $V_T = V_T \cup \{u\}$ 
    for every vertex  $v$  in  $V - V_T$  that is a neighbour of  $u$  do
        if  $d(u) + w(u,v) < d(v)$  then    // a shorter path is found
            set  $d(v) = d(u) + w(u,v)$  and  $p(v) = u$ 
    end
```

Lec 9: Dynamic programming in divide and conquer algorithms

Fibonacci numbers

斐波那契数列是一个递归定义的数列，因此与之相关的问题都很容易联想到动态规划解法。

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Problem: get the n th number in a Fibonacci number sequence.

Recursive version:

```
Procedure F(n)
    if  $n==0$  or  $n==1$  then
        return 1
    else
        return  $F(n-1) + F(n-2)$ 
```


Time Complexity: $O(2^n)$

DP version:

```
Procedure F(n)
  Set A[0] = A[1] = 1
  for i = 2 to n do
    A[i] = A[i-1] + A[i-2]
  return A[n]
```

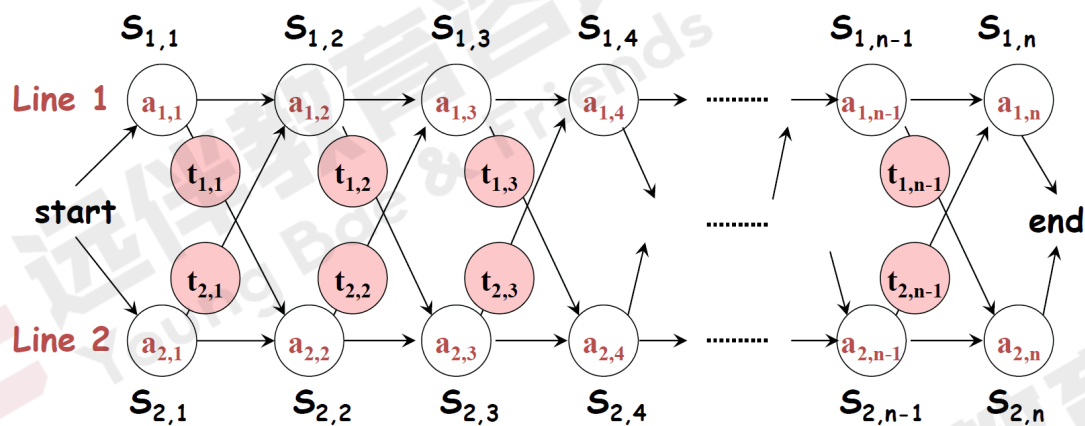
Time Complexity: $O(n)$

在递归的版本中，有大量数值被重复计算了，因此其复杂度远高于dp版本。这个例子说明，dp的一个特点就是会**存储状态**，具体来说，每解决一个子问题，其状态会被存储。

Assembly line scheduling

2 assembly lines, each with n stations ($S_{i,j}$: line i station j)

$S_{1,j}$ and $S_{2,j}$ perform same task but time taken is different



How to determine the best stations to go? There are altogether 2^n choices of stations.

1. If we can compute the fastest ways to get thro' station $S_{1,n}$ and $S_{2,n}$, then the faster of these two ways is the overall fastest way.
2. To compute the fastest ways to get thro' $S_{1,n}$ (similarly for $S_{2,n}$), we need to know the fastest way to get thro' $S_{1,n-1}$ and $S_{2,n-1}$
3. In general, we want to know the fastest way to get thro' $S_{1,j}$ and $S_{2,j}$, for all j .

sub-problems:

- given j , what is the fastest way to get thro' $S_{1,j}$
- given j , what is the fastest way to get thro' $S_{2,j}$

Definitions:

- $f_1[j]$ = the fastest time to get thro' $S_{1,j}$
- $f_2[j]$ = the fastest time to get thro' $S_{2,j}$

The final solution equals to $\min \{f_1[n], f_2[n]\}$

Task:

Starting from $f_1[1]$ and $f_2[1]$, compute $f_1[j]$ and $f_2[j]$ incrementally

Pseudo code:


```

set  $f_1[1] = a_{1,1}$ 
set  $f_2[1] = a_{2,1}$ 
for  $j = 2$  to  $n$  do
  begin
    set  $f_1[j] = \min ( f_1[j-1] + a_{1,j} , f_2[j-1] + t_{2,j-1} + a_{1,j} )$ 
    set  $f_2[j] = \min ( f_2[j-1] + a_{2,j} , f_1[j-1] + t_{1,j-1} + a_{2,j} )$ 
  end
set  $f^* = \min ( f_1[n] , f_2[n] )$ 

```

Lec 10 Space and Time Tradeoffs

在算法设计中，有一种常见的思路是**用空间换时间** (Space and Time Tradeoffs)，指以更大的空间开销为代价，换取更小的时间复杂度。

设计这类算法有两个大的方向：

- **Input-enhancement:** preprocess the input (or its part) to store some info to be used later in solving the problem
 - Distribution counting sort
- **Pre-structuring:** preprocess the input using a data structure to make accessing its elements easier
 - Horspool algorithm for string searching

Distribution counting sort 分布计数排序

分布计数排序最大的特点是不存在比较 (comparision)

- *Input:* $A[1 \dots n]$, where $A[j] \in \{1, 2, \dots, k\}$.
- *Output:* $B[1 \dots n]$, sorted.
- *Auxiliary storage:* $C[1 \dots k]$.

分布计数算法输入 n 个元素的序列，输出这 n 个元素的有序排列，使用 k (k 为 n 个元素中的最大值) 个元素大小的额外空间。

Pseudo code:

```

for  $i \leftarrow 1$  to  $k$ 
  do  $C[i] \leftarrow 0$ 
for  $j \leftarrow 1$  to  $n$ 
  do  $C[A[j]] \leftarrow C[A[j]] + 1$ 
for  $i \leftarrow 2$  to  $k$ 
  do  $C[i] \leftarrow C[i] + C[i-1]$ 
for  $j \leftarrow n$  downto  $1$ 
  do  $B[C[A[j]]] \leftarrow A[j]$ 
     $C[A[j]] \leftarrow C[A[j]] - 1$ 

```

分布技术排序的思想是在排序之前处理输入，得到小于等于每个元素的元素数量 $C[i] = |\text{key} \leq i|$ ，这个数量数值就应该是这个元素在有序序列中的位置。

Time Complexity: $O(n)$

Horspool's Algorithm

Horspool算法是一种基于后缀的匹配方法，它的主要难点在于如何安全地移动窗口，以避免遗漏可能的成功匹配。

Horspool's algorithm is a simple string searching algorithms based on the input enhancement idea of preprocessing the pattern.

- preprocesses pattern to generate a shift table that determines how far to shift the pattern when a mismatch occurs
- always makes a shift based on the text's character c aligned with the last character in the pattern according to the shift table's entry for c

shift table 移位表

Horspool's Algorithm 的预处理结果是一张移位表，记录了根据匹配时目标文本(text)和模式(pattern)对齐的最后一位在模式中出现的位置而得出的至少需要将模式向后移多少位才有可能匹配成功的表格。

For a pattern $P[0..m-1]$, shift size $s(c)$ of a letter c in the text can be precomputed as following:

- if c is in $P[0..m-2]$
 $s(c)$ = The number of characters from c 's rightmost occurrence in $P[0..m-2]$ to the right end of the pattern $P[0..m-1]$
- if c is not in $P[0..m-2]$
 $s(c)$ = the length m of the pattern $P[0..m-1]$, otherwise

Algorithm ShiftTable($P[0..m-1]$)

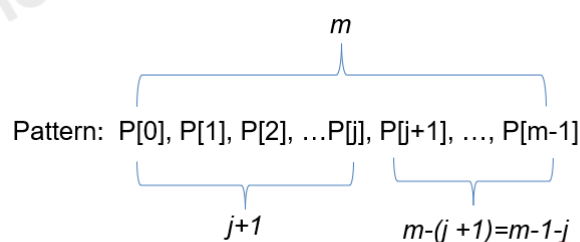
Pseudo code of constructing shift table:

```
Algorithm ShiftTable( $P[0..m-1]$ )
//Fills the shift table used by Horspool's algorithm
//Input: Pattern  $P[0..m-1]$  and an alphabet of possible characters
//output: Table[0..size-1] indexed by the alphabet's characters and filled
//with the //shift sizes computed as before.

Initialize all the elements of Table with m

for  $j=0$  to  $m-2$  do Table[ $P[j]$ ] =  $m-1-j$ 

Return Table
```



Pseudo code of Horspool's Algorithm :

ALGORITHM *HorspoolMatching*($P[0..m-1]$, $T[0..n-1]$)

```
//Implements Horspool's algorithm for string matching
//Input: Pattern  $P[0..m-1]$  and text  $T[0..n-1]$ 
//Output: The index of the left end of the first matching substring
//      or -1 if there are no matches
ShiftTable( $P[0..m-1]$ ) //generate Table of shifts
 $i \leftarrow m-1$  //position of the pattern's right end
while  $i \leq n-1$  do
     $k \leftarrow 0$  //number of matched characters
    while  $k \leq m-1$  and  $P[m-1-k] = T[i-k]$  do
         $k \leftarrow k+1$ 
    if  $k = m$ 
        return  $i-m+1$ 
    else  $i \leftarrow i + \text{Table}[T[i]]$ 
return -1
```

Time Complexity:

- The worst case complexity: $O(mn)$
- for random texts, it is in $O(n)$

Lec 11 Bellman-ford Algorithm and Floyd's Algorithm

这两个算法都是寻找图中最短路径的算法，在学习这两个算法之前，我们需要先熟悉一些概念：

Single-source shortest-paths 单源最短路径

- Consider a (un)directed connected graph G with The edges are labelled by weight
- Given a particular vertex called the **source**
Find **shortest paths** from the source to all other vertices (shortest path means the total weight of the path is the smallest)

All-pairs shortest paths 任意两点间最短路径

- Find **shortest path** between each pair of vertices.

Nonnegative edge weights:

- Dijkstra's algorithm $|V|$ times: $O(VE + V^2 \lg V)$

General

- Bellman-Ford $|V|$ times : $O(V^2 E)$
- Dynamic Programming

Negative Weighted Edges 负权边

在之前学习的图算法中，带权图的权值都是正数，例如 Dijkstra's Algorithm 就只适用与权为正数的图。

Negative-weight Cycles 负环

基于负数权重的边，衍生出另一个概念 **Negative-weight Cycles**。就是在一个闭环中，所有边的权值相加小于 0。

如果一张图存在负环，那么这张图中一些点之间可能不存在最短路径 (只要在负环里一直走下去，路径的权值之和就会一直减小，因此不存在“最短”路径)。

Bellman-Ford algorithm

Bellman-Ford algorithm: Finds all shortest-path lengths from a source $s \in V$ to all $v \in V$ or determines that a negative-weight cycle exists.

Pseudo codes:

```
Algorithm Bellman-Ford( $G=(V, E), s$ )
//input: a graph  $G=(V, E)$  with a source vertex  $s$ 
//output: a array  $d[0..|V|-1]$ , indexed with  $V$ ,  $d[v]$  is the
//length of shortest path from  $s$  to  $v$ 
 $d[s] \leftarrow 0$ 
for each  $v \in V - \{s\}$ 
  do  $d[v] \leftarrow \infty$ 

for  $i \leftarrow 1$  to  $|V| - 1$ 
  do for each edge  $(u, v) \in E$ 
    do if  $d[v] > d[u] + w(u, v)$ 
      then  $d[v] \leftarrow d[u] + w(u, v)$ 

for each edge  $(u, v) \in E$ 
  do if  $d[v] > d[u] + w(u, v)$ 
    then report that a negative-weight cycle exists
```

All-pairs shortest paths:

Problem:

A weighted (di)graph $G = (V, E)$, find a $|V| \times |V|$ matrix, its entry d_{ij} is the shortest-path length between vertices i, j , where $i, j \in V$

解决这个问题的一种原始的方法是对图中的每一个节点应用 Bellman-Ford algorithm,

这样做的复杂度是: $O(V^2 E)$

Dense graph $\implies O(V^4)$ time, e.g., for a complete graph, $|E| = \frac{|V|(|V|-1)}{2}$

Dynamic programming:

动态规划的关键是状态转移方程

Index the sub-problems so that they can be **stored** and **retrieved** easily in a table (i.e., array)

Fill the table in some **bottom-up** manner; start filling the solution of the smallest problem.

在状态转移方程确定的情况下，动态规划可以自底向上解决问题。

Floyd's Algorithm: All Pairs Shortest Paths

Floyd's Algorithm 是一种动态规划算法

On the k -th iteration, the algorithm determines shortest paths between every pair of vertices i, j that use only vertices among $1, \dots, k$ as intermediate

Floyd's Algorithm 的状态转移方程为:

$$D^{(k)}[i, j] = \min\{D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j]\}$$

$D^{(k)}$ 表示第 k 次迭代之后的结果矩阵(下图伪代码中的最外层循环)。

Pseudo codes:

将图中的每个节点用数字编号, 形成一个 $n \times n$ 的方阵。

ALGORITHM Floyd($W[1..n, 1..n]$)

//Implements Floyd's algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

Time Complexity: $O(n^3)$

Transitive closure of a directed graph 有向图的传递闭包

从数学上来说, 传递闭包是在集合 X 上求包含关系 R 的最小传递关系。从关系图的角度来说, 就是如果原关系图上有 i 到 j 的路径, 则其传递闭包的关系图上就应有从 i 到 j 的边。通俗地讲, 就是确定每个点是否能到达其他每个点。

而这, 把Floyd最短路算法稍微改一下即可。设 E 是原来的关系矩阵, 则可以这样写:

```
for (int k = 1; k <= n; ++k)
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            if (E[i][k] && E[k][j])
                E[i][j] = 1;
```

以上这段算法其实就是 Warshall's Algorithm

数学语言:

$$R_{ij} = \begin{cases} 1 & \text{if there exists a path from } i \text{ to } j, \\ 0 & \text{otherwise.} \end{cases}$$

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \vee (R_{ik}^{(k-1)} \wedge R_{kj}^{(k-1)}).$$

Lec12: Sequence Alignment by Dynamic Programming

Longest Common Subsequence

最大相同子列问题是一个经典的 dp 问题

LCS problem: given $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, find their LCS.

使用暴力解法:

List all possible subsequences of X , check whether they are also subsequences of Y , keep the longer one each time.

Each subsequence corresponds to a subset of the indices $\{1, 2, \dots, m\}$, there are 2^m .

指数复杂度肯定无法满足大型输入的求解, 例如DNA片段的比对。因此我们需要使用动态规划来优化问题的时间复杂度。

动态规划类的算法工作的流程:

- Initialization 初始化
- Recursive Solution 递归求解
- Trace Back 回溯

dp 求解 LCS 问题, 第一步: 最优子结构

Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$

$Z = \langle z_1, z_2, \dots, z_k \rangle (= Z_k)$ be any LCS of X and Y ,

基于以上前提, 我们可以得出以下几条最基本的结论:

1. if $x_m = y_n$, then $z_k = x_m = y_n$, and Z_{k-1} is the LCS of X_{m-1} and Y_{n-1} .
2. if $x_m \neq y_n$, then $z_k \neq x_m$ implies Z is the LCS of X_{m-1} and Y_n .
3. if $x_m \neq y_n$, then $z_k \neq y_n$ implies Z is the LCS of X_m and Y_{n-1} .

寻找最优子结构就是在寻找递归式地分解问题的办法。

dp 求解 LCS 问题, 第二步: 递归求解LCS的长度

想要能够递归求解, 关键就是写出状态转移方程。

根据第一步中提出的基本结论, 我们可以衍生出以下两个结论:

1. if $x_m = y_n$, find LCS of X_{m-1} and Y_{n-1} , then append X_m .
2. if $x_m \neq y_n$, find LCS OF x_{M-1} and Y_n and LCS of X_m and Y_{n-1} , take which is longer.

let $c[i, j]$ is the length of LCS of X_i and Y_j .

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max\{c[i-1, j], c[i, j-1]\} & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

确定状态转移方程的过程就是使用递归思想分解问题的过程, 状态转移方程本质上就是用于将父问题拆分为子问题的可复用逻辑。

根据以上状态转移方程, 可以写出以下代码

c 是一个 $m \times n$ 的矩阵, 称为 dp table


```

lengthofLCS(i, j):
    if(i ==0 || j ==0)
        c[i,j] = 0;
        return 0
    if (x_i == y_j)
        c[i,j] = lengthofLCS(i-1, j-1) + 1
        return c[i,j]
    return
    c[i,j] = max(lengthofLCS(i-1, j), lengthofLCS(i, j-1))
    return c[i,j]

```

以上这段代码返回的就是 LCS 的长度。

dp 求解 LCS 问题，第三步： 根据LCS的长度，我们可以轻松求得LCS

Pairwise Sequence Alignment Problem

Given:

- a pair of sequence (DNA or protein)

可以从很多角度定义两个DNA序列的相似度，其中有一种定义方法就是通过序列对齐的方式来定义其相似度。

给定两个DNA序列A和B，对齐的方式是将空格分别插入到A和B序列中，得到具有相同长度的对齐后的序列C和D

空格与空格对齐是不允许的

- a method of scoring a candidate alignment

Do:

- find an alignment for which the score is maximized

Approaches:

- Dynamic programming
- Heuristic solutions
 - Dot matrix plot
 - FASTA
 - BLAST

动态规划解法 Global alignment :

Optimal alignment of n bases:

1. Incorporates optimal alignment of 1...n-1 bases
2. Combine with best alignment for n^{th} base

在第n位处的最优对齐序列等于 1 至 n-1 处的最优对齐序列拼上第 n 位出的最优对齐元素。

由于**空格与空格对齐不被允许**，在第 n 位处我们有三种选择:

设 两序列为 x 和 y

1. 两个序列各往后取一位元素对齐 (如果两个序列都未用完)
2. x取下一位元素，y用空格占位对齐
3. y取下一位元素，x用空格占位对齐

想要最后的得分最优，此处应该选择在三种选择的前提下的最优解中的最优解，即为如下状态转移方程：

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

d 为空格与元素对齐的减分 (gap penalty)

F 是dp table，存储结果

s 是 scoring matrix 存储每种对齐方案的得分

如果gap penalty不是一个常量，状态转移方程如下：

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + \gamma(i-k), k=0, \dots, i-1 \\ F(i, j-1) + \gamma(i-k), k=0, \dots, i-1 \end{cases}$$

大概的求解步骤：

- Initialize: $F(0,0) = 0$, $F(i,0) = i \times d$, $F(0,j) = j \times d$
- Fill from top left to bottom right using recursive formula
- $F(i, j)$ is the highest score

动态规划解法 Local alignment:

Global alignment are often not effective for highly diverged sequences - do not reflect the biological reality that two sequences may only share limited regions of conserved sequence.

Global methods are useful when you want to force two sequences to align over their entire length

Local alignment 和 Global alignment 的解法基本思路是相同的，只不过在Local alignment中如果在某一位处所有对齐方法的最优解都是负数，我们会认为舍弃这一位前面的部分 (得分为0) 更好。基于这样的思路，状态转移转移方程为：

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + \gamma(i-k), k=0, \dots, i-1 \\ F(i, j-1) + \gamma(i-k), k=0, \dots, i-1 \\ 0 \end{cases}$$

如果在某一步中选择了 0 就意味着这一步中的 i, j 之前的部分被从各自的序列中舍去了。

local alignment 与 global alignment 在编码时很大的一个区别就是初始化方式不同：

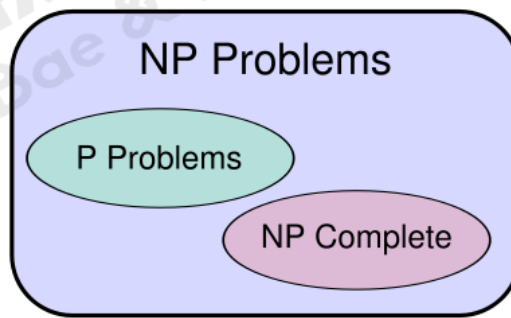
- Initialize: $F(0,0) = 0$, $F(i,0) = 0$, $F(0,j) = 0$

从dp table 中每一个不为0的位开始 trace back，遇到 0 停止，就会形成可能不止一个的 Local alignment 值。

Lec 13: Introduction to Computational Complexity Theory

Hard Computational Problems

尽管算法能简化很多问题的求解，但是算法对效率的优化依然是存在极限的，这一类可以被证明难以计算的问题被称为 NP 问题



当一个算法能在**多项式时间复杂度**内解决某一类问题时，我们就认为其是一个高效的算法，但是也存在以一类非常难以计算求解的问题，我们没法设计出一个在多项式时间复杂度内解决这些的问题的算法，这一类问题就被称作 NP 问题（NP Problems）。

P/NP问题是一个在理论信息学中计算复杂度理论领域里至今未被解决的问题，我们至今无法证明 $NP = P$ or $NP \neq P$ 。

下面我们举一些 NP 问题的例子

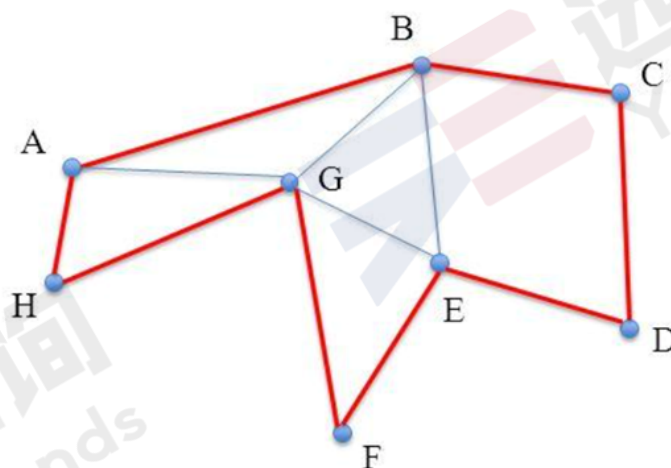
Hamiltonian Circuit 哈密顿电路

Input: A connected graph G

Question: Does G have a Hamiltonian circuit? i.e., does G have a circuit that passes through every vertex exactly once, except for the starting and ending vertex?

Hamiltonian Circuit

A circuit that passes through every vertex at most once is called a **Hamiltonian circuit**.



A - H - G - F - E - D - C - B - A

Knapsack Problem 背包问题

Input: Given n items with integer weights $w_1, w_2, \dots, w_n$ and integer values $v_1, v_2, \dots, v_n$, and a knapsack with capacity W.

(taking fractional parts of items is NOT allowed)

Circuit-SAT

Question: is there an assignment of values to the inputs so that the output value is 1?

- A **decision** problem is a computational problem for which the output is either **yes** or **no**.
- In an **optimisation** problem, we try to **maximise** or **minimise** some value.
- An optimisation problem can be **turned into** a decision problem if we add a **parameter k**; and then ask whether the optimal value in the optimisation problem is **at most** or **at least k**.
- Note that if a decision problem is **hard**, then its related optimisation version must also be **hard**.

Decision problem: For any integer k , is there a subset of items whose total weight does not exceed W and whose total value is **at least** k ?

undecidable problems

not all Decision Problems Be Solved By Algorithms.

The problems can not solved by algorithms is called undecidable problems.

One such a problem is **Halting Problem**

Halting Problem:

Given a computer program and an input to it, determine whether the program will halt on that input or continue working indefinitely on it

Solving/Verifying a problem

解决问题和验证问题是两个不同的概念。当一个问题的可能的解甚至无法被验证时，就更不用谈解决了，但是我们想要验证一个问题的解是否正确不一定要知道问题如何解决。

Solving a problem is different from verifying a problem:

- **solving**: we are given an input, and then we have to FIND the solution
- **verifying**: in addition to the input, we are given a "**certificate**" and we verify whether the certificate is indeed a solution

We may not know how to solve a problem efficiently, but we may know how to verify whether a candidate is actually a solution

Lec 14: Introduction to Computational Complexity Theory

P/NP problem

- **The complexity class P** is the set of all decision problems that can be **solved** in worst-case **polynomial time**.
- **The complexity class NP** is the set of all problems that can be **verified** in **polynomial time**.

P stands for polynomial, and NP stands for non-deterministic polynomial.

Non-deterministic machine is a model of computation, and can be used to model verification.

The Class P

MST problem is in P

- Given a weighted graph G and a value k , does there exists an MST with weight at most k ?
- run Kruskal's algorithm (polynomial time) and if the MST found has weight at most k , then the answer is "Yes"

Single-source-shortest-paths problem is in P

- Given a weighted graph G , a source vertex s , and a value k , does there exist shortest paths from s to every other vertex whose path length is at most k ?
- run Dijkstra's algorithm (polynomial time) and if the paths found have lengths at most k , then answer is "Yes"

The Class NP

Hamiltonian circuit problem is in NP

- we can check in polynomial time if a proposed circuit is a Hamiltonian circuit

0/1 Knapsack problem is in NP

- we can check in polynomial time if a proposed subset of items whose weight is at most W and whose value is at least k

Circuit-SAT is in NP

- we can check in polynomial time if proposed values lead to a final output value of 1

P = NP?

Note that $P \subseteq NP$, but mathematicians and computer scientists do not know whether $P = NP$ or $P \neq NP$

However, there is a common belief that P is different from NP , which means there is some problem in NP that is not in P .

Is an $O(n)$ algorithm efficient?

How about $O(n \log n)$?

$O(n^2)$?

$O(n^{10})$?

$O(n^{\log n})$?

$O(2^n)$?

$O(n!)$?

polynomial time

$O(n^c)$ for some constant c

non-polynomial time

Polynomial-time reduction

Given any two decision problems A and B , we say that:

- (1) A is polynomial time reducible to B , or
- (2) there is a polynomial time reduction from A to B

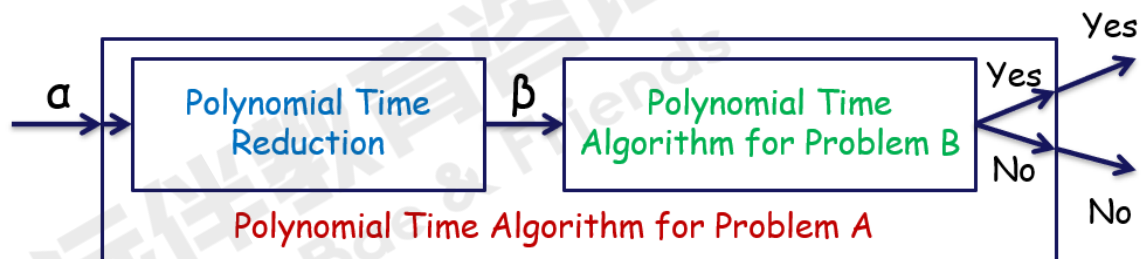
if given any input α of A , we can **construct** in **polynomial time** an input β of B such that α is yes if and only if β is yes.

We use the notation $A \leq_p B$

Intuitively, this means that problem B is at least as difficult as problem A

Reduction: a transformation between instance α of Problem A and instance β of Problem B such that

- The transformation takes **polynomial time**
 - Polynomial in size of the input instance
- The answer for α is "YES" **if and only if** the answer for β is also "YES"



If problem A is **reducible** to problem B in polynomial time, which means Problem B is at least **as hard as** Problem A.

Two Ways to Use Reductions

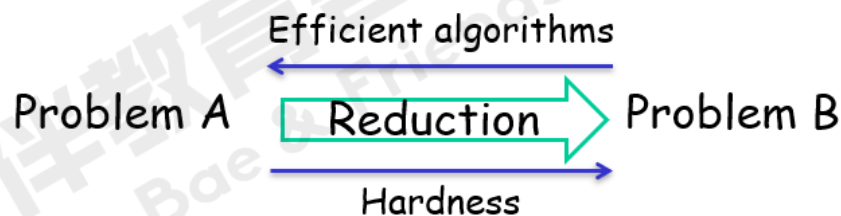
Suppose Problem A is reducible to Problem B

Solve problem

- If there exists **efficient algorithm** for Problem B, then we can solve Problem A efficiently

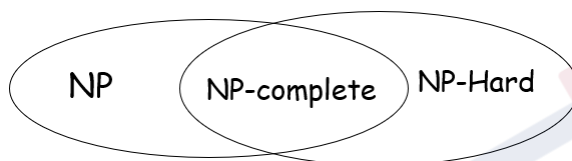
Prove Hardness

- If Problem A is hard, then Problem B is also hard



NP-hardness

- A problem M is said to be **NP-hard** if every other problem in NP is polynomial time reducible to M
 - intuitively, this means that M is at least as difficult as all problems in NP
- M is further said to be NP-complete if



- M is in NP, and
- M is NP-hard

NP-Completeness

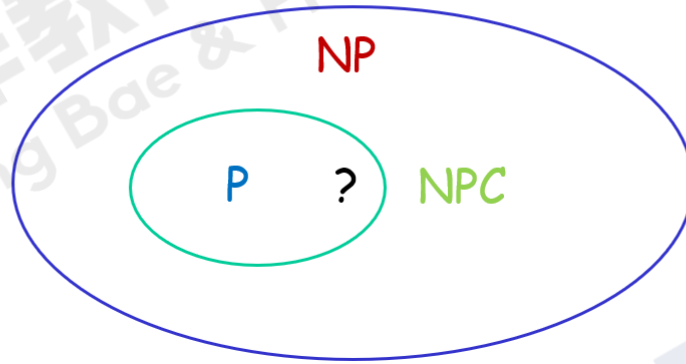
Problem A is NP-complete if:

1. Problem A is **in NP**
2. For **any** Problem A' in NP, A' is **reducible** to A in polynomial time

Class NPC: The class of all NP-complete problems, which is a subclass of NP

- The **hardest** problems in NP
- Solve **a** problem in NPC, you can solve **ALL** problems in NP

If $NP \cap P$ is not empty, then $NP = P$



NP-Complete Problem

- The **Cook-Levin Theorem** states that Circuit-SAT is NP-complete (a "first" NP-complete problem)
- Using polynomial time reducibility we can show existence of other NP-complete problems

A useful result to prove NP-completeness:

$$\text{If } L1 \leq_P L2 \text{ and } L2 \leq_P L3, \text{ then } L1 \leq_P L3$$

Proof of NP-Completeness

Given a Problem A, prove that A is NP-complete:

Proof Scheme 1:

- Show Problem A is in **NP** (easier part)
- For all Problems in **NP**, reduce them to A in **polynomial time**
 - This has been done for 3SAT, the first NP-complete problem

Proof Scheme 2:

- Show Problem A is in **NP** (easier part)
- For arbitrary problem A' in **NPC**, reduce A' to A in **polynomial time**
 - This would be much easier

NP-Complete Problems example:

- Hamiltonian Circuit Problem
- 0/1 Knapsack Problem
- Circuit-SAT
- CNF-SAT and 3-SAT (conjunctive normal form satisfiability problem)
- Vertex Cover

Lec 15: Coping with the Limitations of Algorithm Power

There are two principal approaches to tackling difficult combinatorial problems (NP-hard problems):

- Use a strategy that guarantees solving the problem exactly but doesn't guarantee to find a solution in polynomial time
- Use an approximation algorithm that can find an approximate (sub-optimal) solution in polynomial time

Exact Solution Strategies

- *exhaustive search* (brute force) 穷举法
 - useful only for small instances
- *backtracking* 回溯
 - eliminates some unnecessary cases from consideration 剪枝
 - yields solutions in reasonable time for many instances but worst case is still exponential
- *branch-and-bound*
 - further refines the backtracking idea for optimization problems
- *dynamic programming*
 - applicable to some problems (e.g., the knapsack problem)

Backtracking 回溯

- Construct the *state-space tree*
 - nodes: partial solutions
 - edges: choices in extending partial solutions
- Explore the state space tree using depth-first search
- “Prune” *nonpromising nodes*

dfs stops exploring subtrees rooted at nodes that cannot lead to a solution and backtracks to such a node's parent to continue the search

n 皇后问题

Place n queens on an n -by- n chess board so that no two of them are in the same row, column, or diagonal

回溯的具体做法是：使用一个数组记录每行放置的皇后的列下标，依次在每一行放置一个皇后。每次新放置的皇后都不能和已经放置的皇后之间有攻击：即新放置的皇后不能和任何一个已经放置的皇后在同一列以及同一条斜线上，并更新数组中的当前行的皇后列下标。当 N 个皇后都放置完毕，则找到一个可能的解。当找到一个可能的解之后，将数组转换成表示棋盘状态的列表，并将该棋盘状态的列表加入返回列表。

使用回溯法解决 n 皇后问题的时间复杂度为 $O(n!)$

从这个例子就可以看出“回溯”的含义就是在遇到不可行情况后回到上一步，或者在找到解后回到上一步。

Comments on Backtracking

Typically, backtrack is applied to difficult combinatorial problems for which no efficient algorithms for finding exact solutions possibly exist.

Unlike exhaustive search approach, which is doomed to be extremely slow for all instances of a problem, backtrack at least holds a hope for solving some instances of nontrivial size in an acceptable amount of time.

Even if backtracking does not eliminate any elements of a problem's state space and ends up generating all its elements, it provides a specific technique for doing so, which can be of value in its own right.

Branch-and-Bound 分支和边界

- An enhancement of backtracking
- Applicable to optimization problems
- For each node (partial solution) of a state-space tree, computes a bound on the value of the objective function for all descendants of the node (extensions of the partial solution)

Uses the bound for:

- ruling out certain nodes as “nonpromising” to prune the tree – if a node’s bound is not better than the best solution seen so far
- guiding the search through state-space

The traveling salesperson optimization problem (TSP)

- Given a graph, the TSP Optimization problem is to find a tour, starting from any vertex, visiting every other vertex and returning to the starting vertex, with minimal cost.
- It is NP-complete
- We try to avoid $n!$ exhaustive search by the branch-and-bound technique

Branch-and-Bound 的核心概念就是 lower bound (lb), lower bound 其实就是我们估算出来的一个最优解, 或者说这一支解的下界。如果一个分支解的下边界还没有已近得到的一个解好, 或者说和一个已经得到的解一样好, 那么这个分支也就不再继续算下去了。

lb的计算方式为

$$lb = \Sigma(\text{distances of two nearest cities of } v \text{ in } G)/2$$

其实就是每个节点必有一进一出, 因为不能走重复的路, 所以一进一出都选分别与其他节点先相连的最短路径和次短路径就是最理想情况下的最短路径, 最终真实的最短路径的长度很可能比初始的 lb 大。

这个 lb 再走完每一步后都会更新, 对于已经走过的节点, 应该使用已经走过的路径长度计算 lb。

Lec16: Dynamic Programming

1-dimensional DP Example

Example 1

Problem: given n , find the number of different ways to write n as the sum of 1, 3, 4

- Define subproblems:
 - Let D_n be the number of ways to write n as the sum of 1, 3, 4
- Find the recurrence:
 - Consider one possible solution $n = x_1 + x_2 + \dots + x_m$
 - If $x_m = 1$, the rest of the terms must sum to $n - 1$
 - Thus, the number of sums that end with $x_m = 1$ is equal to D_{n-1}
 - Take other cases into account ($x_m = 3, x_m = 4$)
- Recurrence is then

$$D_n = D_{n-1} + D_{n-3} + D_{n-4}$$

- Solve the base cases
 - $D_0 = 1$
 - $D_n = 0$ for all negative n

- Alternatively, can set: $D_0 = D_1 = D_2 = 1$, and $D_3 = 2$

Implementation

```
D[0] = D[1] = D[2] = 1;
D[3] = 2;
for(i = 4; i <= n; i++)
    D[i] = D[i-1] + D[i-3] + D[i-4];
```

Example 2 完美覆盖问题

Problem: Given n , find the number of ways to fill a $3 \times n$ board with dominoes. (board with dominoes are 1×2)

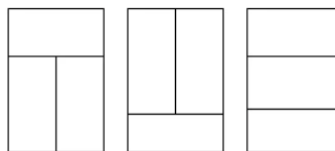
Here is one possible solution for $n = 12$:



以下思路与老师提供的不同

设 $f(n)$ 为需填充区域长为 n 时填充的方法数量。

首先确定, n 为奇数情况下, 无法密铺区域, 只需要考虑偶数情况, 于是考虑 $f(n)$ 和 $f(n-2)$ 的关系, 如果填充区域向右增加 2 个单位, 有以下三种方法填满新增的小区域:



因此 $f(n)$ 与 $f(n-2)$ 的关系中, 至少会有一项 $3f(n-2)$ 。

这道题目有意思的地方在于, $f(n)$ 不仅仅和 $f(n-2)$ 有关系, 请看下面这种铺法:



这种铺法需要 4 个单位, 并存在垂直方向对称的铺法, 于是需填充区域向右扩展 2 个单位时, 我可以选择不铺, 等下一次再扩展两个的单位, 也就就是一共扩展 4 个单位时, 再用以上这种铺法填充区域, 因此 $f(n)$ 还和 $f(n-4)$ 有关系, 经过实验观察还可以发现, 想要铺满宽大于 2 的空间 (长为偶数), 有且仅有形如上面的铺法, 也就是只有两种铺法, 那么 $f(n)$ 和 $f(n-4)$ 的关系中至少会有一项 $2f(n-4)$. 推广到所有情况, 其实 4, 6, 8 都是一样的, 也就是说 $f(n)$ 其实和 $2(f(n-4) + f(n-6) + \dots + f(2) + f(0))$ 有关。

那么至此可以写出状态转移方程:

$$f(n) = 3 * f(n-2) + 2 * (f(n-4) + f(n-6) + \dots + f(0))$$

2-dimensional DP

见前文提到的 **Longest Common Subsequence**

Interval DP Example

Problem: given a string $x = x_1 \dots x_n$, find the minimum number of characters that need to be inserted to make it a palindrome

Example:

- x : Ab3bd
- Can get "dAb3bAd" or "Adb3bdA" by inserting 2 characters (one 'd', one 'A')

Define subproblems:

- Let D_{ij} be the minimum number of characters that need to be inserted to make $x_{i\dots j}$ into a palindrome

Find the recurrence:

- Consider a shortest palindrome $y_1\dots y_k$ containing $x_{i\dots j}$
- Either $y_1 = x_i$ or $y_k = x_j$
- $y_{2\dots k-1}$ is then an optimal solution for $x_{i+1\dots j}$ or $x_{i\dots j-1}$ or $x_{i+1\dots j-1}$
 - Last case possible only if $y_1 = y_k = x_i = x_j$

$$D_{ij} = \max \begin{cases} 1 + \min(D_{i+1,j}, D_{i,j-1}) & x_i \neq x_j \\ D_{i+1,j-1} & x_i = x_j \end{cases}$$

Tree DP Example

Problem: given a tree, color nodes black as many as possible without coloring two adjacent nodes

Subproblems:

- First, we arbitrarily decide the root node r
- B_v : the optimal solution for a subtree having v as the root, where we color v black
- W_v : the optimal solution for a subtree having v as the root, where we don't color v
- Answer is $\max(B_r, W_r)$

Find the recurrence:

- Crucial observation: once v 's color is determined, subtrees can be solved independently
- If v is colored, its children must not be colored

$$B_v = 1 + \sum W_u, u \in \text{children}(v)$$

- If v is not colored, its children can have any color

$$W_v = 1 + \sum \max(B_u, W_u), u \in \text{children}(v)$$

Three step of dp

1. Defining subproblems
2. Finding recurrences
3. Solving the base cases

Lec 17: Dynamic Programming (2)

Knapsack 0-1 Problem

The goal is to maximize the value of a knapsack that can hold at most W units (i.e., lbs or kg) worth of goods from a list of items $I_0, I_1, \dots, I_{n-1}$.

Each item has 2 attributes:

- Value – let this be v_i for item I_i
- Weight – let this be w_i for item I_i

0-1 背包问题的特点之一在于，一个物品只有带或者不带两个状态，不存在带一部分的情况。

第一步，找出子问题：

Our first attempt might be to characterize a sub-problem as follows:

- Let S_k be the optimal subset of elements from $\{I_0, I_1, \dots, I_k\}$.
 - What we find is that the optimal subset from the elements $\{I_0, I_1, \dots, I_k\}$ may not correspond to the optimal subset of elements from $\{I_0, I_1, \dots, I_k\}$ in any regular pattern

此处的难点在于 S_{k+1} 下的解未必包含 S_k 的解

请看下面这个例子：

Item	Weight	Value
I_0	3	10
I_1	8	4
I_2	9	9
I_3	8	11

maximum weight the knapsack can hold is 20.

The best set of items from I_0, I_1, I_2 is I_0, I_1, I_2 BUT the best set of items from $\{I_0, I_1, I_2, I_3\}$ is $\{I_0, I_2, I_3\}$

因此物品每增加一个，就必须重新计算最大价值，由此得出状态转移方程为：

- Let $B[k, w]$ represent the maximum total value of a subset S_k with weight w .
- Our goal is to find $B[n, W]$, where n is the total number of items and W is the maximal weight, the knapsack can carry.

$$B[k, w] = \max \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max\{B[k-1, w], B[k-1, w-1] + b_k\} & \text{otherwise} \end{cases}$$

First case: $w_k > w$:

- Item k can't be part of the solution! If it was the total weight would be $> w$, which is unacceptable.

Second case: $w_k \leq w$:

- Then the item k can be in the solution, and we choose the case with greater value

```

for w = 0 to W { // Initialize 1st row to 0's
    B[0,w] = 0
}
for i = 1 to n { // Initialize 1st column to 0's
    B[i,0] = 0
}
for i = 1 to n {
    for w = 0 to W {
        if wi <= w { //item i can be in the solution
            if vi + B[i-1,w-wi] > B[i-1,w]
                B[i,w] = vi + B[i-1,w-wi]
            else
                B[i,w] = B[i-1,w]
        }
        else
            B[i,w] = B[i-1,w] // wi > w
    }
}
}

```

Time complexity: $O(2^n)$

Lec18: Coping with the Limitations of Algorithm Power(2)

面对NP完全问题 (NP-Complete)，我没办法在多项式时间内求出精确解吗，但是换一种思路，我们能在多项式时间内求出NP完全问题的近似解吗？

答案是可以的，先来回顾以下解决NP完全问题的大概思路：

- Brute-force algorithms. 暴力破解
 - Develop clever enumeration strategies.
 - Guaranteed to find optimal solution.
 - No guarantees on running time.
- Heuristics. 启发式
 - Develop intuitive algorithms.
 - Guaranteed to run in polynomial time.
 - No guarantees on quality of solution.

下面这种思路就是本章节新引入的近似解

- Approximation algorithms. 近似算法
 - Guaranteed to run in polynomial time.
 - Guaranteed to find "high quality" solution, say within 1% of optimum.

Obstacle: need to prove a solution's value is close to optimum, without even knowing what optimum value is

Heuristics

启发式算法之所以被称作这个名字，是因为启发式算法的优化（剪枝）思路来源于常识，经验与直觉 (common-sense rule)，并非是通过数学方法证明出来的。

例如 TSP 算法中 lower bound 的算法，以及0-1背包问题中，从 value/weight 值最高的物品开始取

Approximation Algorithms

Find a “good” solution fast

- sufficient for many applications
- we often have inaccurate data to start with, so approximation may be as good as optimal solution

近似算法能快速求出一个相对好的结果（不是最优解），但是对于大部分应用场景来说足够了。除此之外，在实际应用中，由于输入的数据本身也不完全准确，基于这些数据算出来近似解和最优解可能并没有太大差距。

Accuracy Ratio

准确率是衡量近似解和最优解有多接近的参数。

- $f(s_a)$ = value of objective function for solution given by approximation algorithm
- $f(s^*)$ = value of objective function for optimal solution
- $r(s_a)$: Accuracy Ratio of s_a

minimization problems: $r(s_a) = f(s_a)/f(s^*)$

maximization problems: $r(s_a) = f(s^*)/f(s_a)$

in either case $r(s_a) \geq 1$

Performance Ratio

If there exists $c \geq 1$, such that $r(s_a) \leq c$ for all instances of a problem, the given algorithm is called a c -approximation algorithm

The smallest value of c that holds for all instances is called the performance ratio, R_A , of the algorithm

if $c=1.1$, your approximation is never more than 10% worse than optimal

但是在一些情况下，这个误差的上界是无法求取的，如果误差的上界不存在，近似算法在求解一些特殊输入时可能会有相当大的误差。

例如对于TSP问题：

if $P \neq NP$, no c -approximation algorithm for TSP exists

VERTEX-COVER 顶点覆盖

最小顶点覆盖的问题是计算机科学中的经典优化问题，并且是具有近似算法的NP难优化问题的典型示例

Instance: an undirected graph $G = (V, E)$.

Problem: find a set $C \subseteq V$ of minimal size so that for any $(u, v) \in E$, either $u \in C$ or $v \in C$.

Proof: Two vertices outside a vertex-cover cannot be connected by an edge

Pseudo Code:

- $C \leftarrow \phi$
- $E' \leftarrow E$
- **while** $E' \neq \phi$
 - **do** let (u,v) be an arbitrary edge of E'
 - $C \leftarrow C \cup \{u,v\}$
 - **remove** from E' every edge incident to either u or v .
- **return** C .

Time Complexity: $O(n^2)$

$$OPT \leq \text{oursolution} \leq 2 * CV$$

Worst-case is $2*OPT$, so it called 2-Approximation algorithm

TSP 的一种近似解算法

Euclidean Instances

Euclidean instances of the TSP problem obey the natural geometry of a 2D map.

- triangle inequality: $d[i, j] \leq d[i, k] + d[k, j]$
- symmetry: $d[i, j] = d[j, i]$

For Euclidean instances, the nearest neighbor algorithm satisfies:

$$r(s_a) \leq \frac{1}{2}(\log_2 n + 1), \text{ where } n = \# \text{cities}$$

Twice-Around-the-Tree Algorithm

- Stage 1: Construct a minimum spanning tree of the graph (e.g., by Prim's or Kruskal's algorithm)
- Stage 2: Starting at an arbitrary vertex, create a path that goes twice around the tree and returns to the same vertex
- Stage 3: Create a tour from the circuit constructed in Stage 2 by making shortcuts to avoid visiting intermediate vertices more than once

$$OPT \leq MST \text{ Tour} \leq 2 * MST$$

Worst-case is $2*OPT$, so **Twice-Around-the-Tree Algorithm** is a 2-Approximation algorithm